Deeparture V2 - Phase 2A Core Admin V2

Code-Aware Developer Specification - Revision 1.3

Basis of this revision: audit of the uploaded Phase 1 source package `DPHandover_V2.zip`, prepared 30 September 2026.

Important baseline note: the supplied package is a local working-tree snapshot based on branch `main`, base commit `137747914daa2de1a9c4652f08a81a2a3820fea8`, plus uncommitted changes. The package itself states that the exact Render-deployed revision is not yet confirmed. Phase 2A can be developed from this source after the reviewed state is committed, but deployment identity should still be confirmed before it becomes the authoritative staging baseline.

The supplied Phase 1 verification report records 34 local API/browser check groups passing. Phase 2A must preserve those working foundations rather than rebuild them.

Turn the existing Phase 1 backend and starter dashboard into the operational **Deeparture Admin V2** needed to populate, review, publish and manage the catalogue independently of operators.

Core business flow:

```
Deeparture Admin
v
Create / import operator organisation
v
Create / import liveaboard or resort
v
Draft / pending review
v
Admin review + approval
v
Publish
v
Automatic public page
v
Invite operator to claim existing organisation
```

Phase 2A must **extend the supplied codebase**. It must not replace working Phase 1 services merely to match a different project structure.

The following functionality is present in the uploaded source and should normally be reused.

Area	Existing source	Phase 2A treatment
Authentication	`backend/src/v2/middleware/v2Auth.js`, existing login/session flow	**REUSE / EXTEND**
Organisation roles	`organisationMembership.model.js`, `organisationAccess.js`	**REUSE**
Organisation ownership security	ownership checks in `inventory.controller.js`	**REUSE**
Liveaboard/resort models	`vessel.model.js`, `resort.model.js`	**EXTEND, do not replace**
Itineraries/departures/cabins	existing V2 models/controllers	**REUSE**
Resort room/package/rate foundations	existing V2 models/controllers	**REUSE**
Pricing inheritance/overrides/offers	`pricing.service.js` + departure model	**REUSE**
Public auto-generated pages	public controllers/search + Next.js dynamic pages	**REUSE**
Public search	`search.service.js`	**REUSE / later extend**
CSV/XLSX parsing	`bulkFile.service.js`	**REUSE as adapters, refactor pipeline after parse**
Catalogue import logic	`catalogImport.service.js`	**REFACTOR, not discard**
Inventory import logic	`inventoryImport.service.js`	**REFACTOR, not discard**
Operator claim token security	`claim.service.js`, `claimToken.model.js`	**REUSE**
Bulk claim invitations	`adminClaim.controller.js`	**REUSE / add admin UI and single-record actions as needed**
Extensible attributes	attribute definition/product attribute models + service	**REUSE / add admin management UI**
Location model	`location.model.js`	**REUSE / add CRUD and structured references**
Audit foundation	`auditLog.model.js`, `audit.service.js`	**REUSE / broaden coverage**

Area	Existing source	Phase 2A treatment
Email transport safety	SMTP/test transport + `notification.service.js`	**REUSE / split workflow notifications**
Starter dashboard	`frontend/pages/dashboard/v2/index.jsx`	**REUSE concepts only; build separate Admin V2 UI**

Do not rebuild pricing, claims, automatic public page generation, ownership security, or the core organisation/membership model unless a demonstrated defect requires it.

Before the Phase 2A staging deployment is treated as authoritative:

1. Commit the reviewed local Phase 1 fixes.
2. Record the full commit hash.
3. Confirm Render frontend/backend are deployed from that exact commit.
4. Confirm the exact V2 Atlas staging database name and that it is separate from V1 production.
5. Keep secrets only in deployment environment/secret storage.

This is a deployment identity requirement, not a request to rewrite Phase 1.

Use these classifications throughout implementation:

```
REUSE = existing implementation is already suitable
EXTEND = keep existing code/data model and add capability
REFACTOR = preserve behaviour/data but introduce the required service boundary
BUILD NEW = Phase 1 does not contain the required feature
```

The current frontend has a shared `/dashboard/v2` page intended primarily for an operator organisation. It is not the required Admin V2 control centre.

Build a separate responsive Admin V2 area:

```
/admin/v2
/admin/v2/liveaboards
/admin/v2/resorts
/admin/v2/drafts
/admin/v2/pending
/admin/v2/import
/admin/v2/operators
/admin/v2/attributes
/admin/v2/locations
/admin/v2/enquiries
```

Requirements:

- protect `/admin/v2/*` for `admin` users only
- enforce admin permission again in backend APIs
- desktop/tablet/mobile navigation
- loading, empty, permission and error states
- do not make Admin V2 depend on selecting an operator organisation before global catalogue administration is possible

Existing code to extend

- `frontend/src/components/auth/AuthProvider.jsx` currently treats `/dashboard` as protected; extend protection/role handling for `/admin/v2`.
- `frontend/src/components/layout/SiteHeader.jsx` is a starter public/operator header; Admin V2 should have its own layout/navigation.

Build an Admin V2 dashboard with at least:

- total liveaboards
- total resorts
- draft listings
- pending-review listings

- unclaimed organisations
- invited organisations
- claimed organisations
- pending admin enquiries
- recent catalogue/admin activity

The current organisation dashboard endpoint can remain for operators. Do not overload it into the global admin dashboard if a dedicated admin endpoint is clearer.

The supplied backend already supports organisation-scoped create/update for vessels and resorts. Admin permission bypasses organisation-membership restrictions and controls editorial fields.

Phase 2A must add global Admin catalogue management.

Admin must be able to:

- list all liveaboards/resorts with pagination
- search by name
- filter by publication status
- filter by operator organisation
- filter by destination
- filter by claim status
- open/edit record
- preview public page
- approve/publish
- unpublish
- restrict/unrestrict
- archive
- see organisation ownership
- see claim/verification state

Publication-state change required

Current Vessel/Resort models only support:

```
draft
published
archived
```

Phase 2A requires a moderation-aware state. Extend the publication status to support at least:

```
draft
pending_review
published
restricted
archived
```

Keep organisation claim state separate. Do not use claim state to represent publication/moderation.

Public search/detail APIs should continue returning only `published` records.

Important import behaviour change

Current `catalogImport.service.js` publishes imported records by default unless `publish=false` is supplied.

Change this. In Phase 2A, imported catalogue records must default to `draft` or `pending_review`, not directly to public publication. Publication should follow explicit Admin review/approval.

Build queues for:

```
draft
pending_review
```

Admin must be able to:

- open a draft
- see source/import context
- see validation/missing-field issues
- edit/correct data
- preview before publication
- approve/publish
- return/reject/archive
- assign/match organisation

Approval/restriction/publication changes should write audit events.

A basic catalogue page may be published without current departure/availability data. Do not require a complete booking feed before the catalogue record can exist.

Current implementation

The Phase 1 code correctly parses CSV/XLSX and imports records, but the source-shaped rows are passed directly into catalogue/inventory services that write MongoDB models.

Relevant files:

```
backend/src/v2/services/bulkFile.service.js
backend/src/v2/services/catalogImport.service.js
backend/src/v2/services/inventoryImport.service.js
backend/src/v2/controllers/catalogImport.controller.js
backend/src/v2/controllers/inventoryImport.controller.js
```

This is a useful Phase 1 foundation, but it is not the future-proof Phase 2A architecture.

Required Phase 2A architecture

```
CSV adapter ----------\
XLSX adapter ----------\
> Mapping / Normalisation
Future adapters -------/ v
Deeparture Normalized Import DTOs
v
Validation
v
Matching / Duplicate Check
v
Preview
v
Admin Confirmation
v
Catalogue Service
v
Database
```

Required normalized contracts

Create documented source-independent DTOs/interfaces for at least:

```
OrganisationImport
LiveaboardImport
ResortImport
```

The core validation/matching/commit services must use Deeparture field names, not spreadsheet column names.

XLSX / CSV

- XLSX remains the preferred admin/human format.
- CSV remains supported.

- Existing `bulkFile.service.js` may be retained as the low-level parser.
- Spreadsheet mapping belongs in adapter/mapping code.
- Parser/adapter code must not commit database records.

Workbook structure

Provide a documented standard Deeparture workbook. Phase 2A should support either:

1. a multi-sheet workbook containing `Operators`, `Liveaboards`, `Resorts`, **or**
2. an explicitly documented equivalent upload flow that still uses the same normalized DTOs.

The architecture must allow later sheets such as itineraries, departures, prices and availability without replacing the import engine.

The current controllers commit immediately after parsing. Phase 2A must introduce a no-write preview step.

Required flow:

```
Upload
v
Parse / map
v
Normalize
v
Validate / match
v
PREVIEW - no catalogue write
v
Admin confirms
v
Commit
```

Preview classifications:

```
CREATE
UPDATE
DUPLICATE_REVIEW
ERROR
SKIP
```

Preview summary should show counts for each classification.

Admin confirmation is mandatory before commit.

Phase 1 already performs useful matching:

- organisation by email / website / exact name
- vessel/resort within organisation by exact name
- re-import preserves IDs in tested cases

Keep this behaviour as a starting point, but move matching into a dedicated preview/matching layer.

Phase 2A priorities:

Organisation

1. explicit Deeparture organisation ID
2. normalized website/domain
3. exact normalized company name
4. operator contact email
5. manual review

Liveaboard / Resort

1. explicit Deeparture record ID
2. external/provider reference when present
3. organisation + stable slug/reference

4. organisation + normalized name
5. manual review

Ambiguous matches must become `DUPLICATE_REVIEW`. Do not silently merge them.

Phase 2A needs a basic import batch identity/result even though the richer history UI belongs to Phase 2B.

Record at minimum:

```
batchId
sourceType
sourceFilename
createdBy
timestamp
totalRows
createdCount
updatedCount
skippedCount
duplicateReviewCount
errorCount
row results
```

Phase 2B can add richer import-history search, retry tooling and reporting.

Existing organisation/membership architecture is strong and should be reused.

Build Admin controls to:

- list/search/filter organisations
- create an unclaimed operator organisation manually
- edit contact/profile fields
- view liveaboards/resorts owned
- filter by unclaimed/invited/claimed/verified/suspended
- suspend/reactivate organisation
- open claim action

One organisation must continue to own both liveaboards and resorts.

Use the existing `suspended` organisation state rather than introducing a second equivalent state unless there is a clear business need.

The existing claim implementation already provides:

- admin-only invitation flow
- random token
- SHA-256 token storage
- expiry
- single-use consumption
- existing-user password verification
- organisation membership attachment
- preservation of existing listings

Do not rewrite this flow.

Phase 2A needs:

- Admin button/action to send a claim invitation to one organisation
- clear status display: unclaimed / invited / claimed / verified
- resend behaviour using the existing token replacement logic
- useful success/error state in Admin UI

Detailed invitation history/revoke management can remain Phase 2B.

The attribute architecture is already extensible.

Reuse:

```
AttributeDefinitionV2
ProductAttributeV2
attribute.service.js
```

Build Admin management UI/API support to:

- list active and inactive definitions
- create
- edit
- activate/deactivate
- order/sort
- set appliesTo
- set data type/options
- set filterable/public-visible flags

Do not add routine facility/accessibility/dietary fields directly to every Vessel/Resort schema.

`LocationV2` already exists but there is no complete Admin CRUD workflow and several current catalogue/import fields still rely on slugs or embedded free-text port objects.

Build Admin CRUD for:

```
country
region
destination
port
city (if retained)
```

Transitional data strategy

Do not break working public search during Phase 2A.

Use a transition such as:

- add structured Location references/IDs to the relevant catalogue/itinerary fields
- continue maintaining existing slug fields where current search depends on them
- dual-write or materialize slugs from canonical Location records

This allows Phase 2A to normalize location identity without forcing the entire Phase 4 customer search redesign now.

Avoid creating duplicate canonical locations for case/spelling variants.

Current Phase 1 behaviour

`enquiry.controller.js` currently:

1. creates the enquiry
2. calls `sendEnquiryNotifications`
3. sends to operator + admin + customer together
4. sets status to `sent_to_operator`

That is **not** the required Deeparture business workflow.

Required Phase 2A flow

```
Customer submits enquiry
v
Enquiry saved as pending admin review
v
Customer acknowledgement
v
Admin notification / Admin V2 queue
v
Admin approves
v
Operator receives enquiry
v
Operator accepts/responds
v
Admin confirms/finalises as configured
```

Required backend changes

Refactor notifications into separate actions rather than one combined function.

For example:

```
sendCustomerEnquiryAcknowledgement()
sendAdminNewEnquiryNotification()
sendOperatorApprovedEnquiry()
```

Add Admin endpoints/actions for at least:

```
approve
reject/spam
confirm/close as appropriate
```

Use explicit valid status transitions. Avoid contradictory Boolean flags.

Preserve enquiry persistence even if email delivery fails.

Current Phase 1 behaviour

Availability is functional but source-specific write logic is embedded in:

```
inventoryImport.service.js
inventory.controller.js
DepartureV2.availability[]
ResortAvailabilityV2
```

For liveaboards, departure import currently replaces the `availability` array directly. Resort availability uses a direct `findOneAndUpdate`.

This must be refactored before external live feeds are added later.

Required Phase 2A boundary

Create a common availability service, for example:

```
backend/src/v2/services/availability.service.js
```

Conceptual API:

```
applyAvailabilityUpdate(normalizedUpdate)
getEffectiveAvailability(inventoryIdentity)
```

All Phase 2A availability writes should route through this service:

```
Admin manual update
CSV/XLSX import
existing inventory controllers
```

Future sources will later include:

```
Operator Portal
API / JSON
Webhook
Google Sheet
XML/SFTP
specialist liveaboard provider
```

Normalized availability update

At minimum support:

```
organisationId
productType
vesselId / resortId
departureId (liveaboard)
cabinTypeId or roomTypeId (optional depending inventory)
availabilityStatus
quantityAvailable (optional)
quantityTotal (optional)
sourceType
sourceReference (optional)
sourceUpdatedAt
receivedAt
```

Status should support at least:

```
unknown
available
limited
sold_out
on_request
```

Resort-specific stop-sell behaviour may remain supported.

Source metadata model change

Current liveaboard cabin availability subdocuments do not retain source metadata per inventory item. Extend the stored availability/effective representation so the source and update timestamp for the applicable inventory can be returned.

Do not label a value `live` simply because it exists.

Stable upsert

Repeated updates for the same inventory must update the intended inventory state rather than create uncontrolled duplicates.

For liveaboards, use stable IDs such as:

```
departureId + cabinTypeId
```

Do not use vessel/cabin names as the long-term identity key.

Phase 2A resolution rule

A simple effective rule is acceptable in 2A, such as latest accepted update, provided:

- consumers read effective availability through the service
- source metadata is retained
- the service can later add source priority/conflict/override logic in Phase 2B

Do not build external provider connectors in Phase 2A.

Admin should be able to view/update test departure availability through the normalized service.

Minimum Phase 2A Admin display:

- departure
- cabin type
- availability status
- spaces/cabins available when known
- source type
- last updated timestamp

The purpose is to prove the service boundary. Advanced freshness/conflict/history UI belongs to Phase 2B.

The Phase 1 audit service/model exists but is currently used only in limited paths.

In Phase 2A ensure audit records are created for at least:

- listing create/edit where appropriate
- publish/unpublish/restrict/archive
- organisation suspend/reactivate
- claim invitation sent
- import commit
- enquiry admin approval/rejection

Do not put passwords, raw claim tokens or secrets into audit payloads.

Full searchable audit UI remains Phase 2B.

Reuse the current safe test/live transport configuration.

Phase 2A must split business notifications so operator email is not sent before Admin approval.

Required paths:

- customer enquiry acknowledgement
- admin new-enquiry notification
- admin-approved enquiry -> operator
- claim invitation -> operator
- listing approval/review email where applicable

Keep staging in safe/test delivery mode unless performing a controlled delivery test.

Production provider/hosting choice remains a release item and does not require replacing the Phase 2A business workflow.

Phase 1 starter pages passed local viewport checks. Preserve that standard.

Every new Admin V2 screen must be designed for:

- desktop
- tablet
- phone

Do not simply shrink wide desktop tables.

Use:

- card/list transformation on mobile
- drawer navigation
- filter sheet/drawer
- touch-friendly actions
- stacked forms

Complex bulk-import review is allowed to be easiest on desktop, but routine review/publish/restrict/enquiry actions must remain usable on phone.

```
1. Establish committed/deployed Phase 1 baseline
2. Admin V2 auth/layout/navigation
3. Admin dashboard APIs/UI
4. Global liveaboard/resort catalogue APIs/UI
5. Publication moderation states + draft/pending workflow
6. Normalized catalogue DTOs/adapters
7. Import preview + confirm + basic batch results
8. Duplicate matching/review classification
9. Organisation Admin UI
10. Claim invitation Admin UX
11. Attribute Admin UI
```

```
12. Location CRUD + structured-reference transition
13. Enquiry Admin-first workflow
14. Normalized Availability Service
15. Admin availability proof UI
16. Broaden audit events
17. Responsive/mobile pass
18. Automated Phase 2A regression + acceptance tests
```

Unless a specific defect is demonstrated, do not rebuild:

- V2 auth/JWT foundation
- organisation membership/role model
- organisation ownership checks
- pricing inheritance/materialization
- existing itinerary/departure/resort data models except required extensions
- claim token generation/hash/expiry/single-use flow
- dynamic public liveaboard/resort pages
- public search foundation
- existing CSV parser / ExcelJS parser as low-level file parsers
- attribute-definition architecture

Not required for Phase 2A:

- full import history/retry dashboard
- advanced duplicate merge workspace
- in-app notification centre
- full searchable audit UI
- source priority/conflict arbitration for multiple availability feeds
- availability history/override expiry UI
- real sharc/Bookinglayer/operator API connector
- automatic website extraction
- scheduled synchronization workers
- polished Operator Portal
- customer marketplace redesign
- PWA/native app
- Agent Portal

These remain Phase 2B or later.

Phase 2A is complete when the developer can demonstrate on staging:

```
Admin login
v
Create/import unclaimed organisation
v
Import one liveaboard + one resort as draft/pending
v
Preview classification before any commit
v
Confirm import
v
Review/edit records
v
Approve/publish
v
Automatic public pages appear
v
Send operator claim invitation
v
Operator claims existing organisation
v
```

```
No duplicate organisation/listing
v
Customer enquiry enters Admin queue first
v
Admin approves -> operator receives enquiry
v
Admin updates a departure's availability through normalized Availability Service
v
Public/search API receives effective availability + source/update metadata
```

All critical Phase 1 regression tests must continue passing.

Add/update repository technical notes documenting:

Import contract

```
supported adapters
normalized import DTOs
mapping boundary
validation/matching service
preview result format
commit/catalogue service
future-adapter entry point
```

Availability contract

```
normalized availability update schema
stable inventory identity
Availability Service entry point
source metadata
Phase 2A effective-resolution rule
consumer response format
future-adapter entry point
Phase 2B deferred conflict/freshness rules
```

Deployment

```
staging branch/commit
frontend URL
backend URL
Atlas staging database name (name only)
known issues
```

After this code-aware Phase 2A specification passes its acceptance checklist, proceed to **Phase 2B - Admin Automation & Management Enhancements**.